

Git - Hoja de Referencia Rapida

Adrian Murciego Lobato | POO - UTAMED | 2025-2026

1. CONFIGURACION INICIAL

git config --global user.name "Tu Nombre"

Establece tu nombre de autor para todos los repositorios.

git config --global user.email "tu@email.com"

Establece el correo electronico asociado a tus commits.

git config --list

Muestra toda la configuracion actual de Git.

2. INICIO / CLONADO

git init

Crea un repositorio Git nuevo en la carpeta actual.

git clone <url>

Descarga un repositorio remoto completo a tu maquina.

3. CICLO DE VIDA (status, add, commit, log)

git status

Muestra que archivos estan modificados, en staging o sin seguimiento.

git add <archivo> | git add .

Pasa archivos al area de staging, preparandolos para el commit.

git commit -m "mensaje"

Crea una instantanea permanente con los cambios en staging.

git log | git log --oneline

Lista el historial de commits. --oneline lo muestra compacto.

4. SINCRONIZACION (push, pull)

git push origin <rama>

Sube los commits locales al repositorio remoto.

git pull origin <rama>

Descarga y fusiona los cambios remotos con tu rama local.

git remote -v

Muestra los repositorios remotos configurados.

5. GESTION DE RAMAS (branch, checkout, merge)

git branch

Lista las ramas locales. La activa aparece marcada con *.

git branch <nombre>

Crea una nueva rama sin cambiar a ella.

git checkout <rama> | git switch <rama>

Cambia a la rama indicada. switch es la alternativa moderna.

git checkout -b <rama> | git switch -c <rama>

Crea la rama y cambia a ella en un solo paso.

git merge <rama>

Fusiona los cambios de <rama> en la rama actual.

git branch -d <rama>

Elimina una rama local ya fusionada.

6. DESHACER CAMBIOS

git checkout -- <archivo>

Descarta cambios locales de un archivo (vuelve al ultimo commit).

git reset HEAD <archivo>

Saca un archivo del staging sin perder los cambios.

git revert <hash>

Crea un nuevo commit que deshace los cambios del indicado.

7. FLUJO DE TRABAJO RECOMENDADO

1. git checkout -b feature/mi-funcionalidad

Crea una rama para tu funcionalidad (nunca trabajes en main).

2. (editas, adds, commits incrementales)

Haz commits pequenos y frecuentes con mensajes descriptivos.

3. git push origin feature/mi-funcionalidad

Sube tu rama al remoto para respaldo y revision.

4. git checkout develop && git merge feature/...

Fusiona en develop. Resuelve conflictos si los hay.

5. git push origin develop

Sube develop actualizado al remoto.

8. BUENAS PRACTICAS

Mensajes: feat: / fix: / docs: / refactor:

Usa prefijos convencionales para commits claros y buscables.

.gitignore

Excluye .class, .idea/, out/, target/ y archivos generados.